Advanced Quantitative Research Methods

Chapter 6: Linear Model Selection and Regularisation

Prof. Dr. Christoph Weisser

Slides, lab notebooks and data:
github.com/ChrisW09/Advanced-Quantitative-Research-Methods

The course at a glance

Chapter	Topic
Ch. 1	Introduction
Ch. 2	Statistical learning — accuracy, bias–variance
Ch. 3	Linear regression — estimation, inference, diagnostics
Ch. 4	Classification — logistic regression, confusion matrix, ROC
Ch. 5	Resampling — validation, CV, bootstrap
Ch. 6	**Model selection and regularisation** — ridge, lasso
Ch. 7	Moving beyond linearity — splines, GAMs
Ch. 8	Tree-based methods — trees, forests, boosting

Contents

1. Why this chapter matters
2. Subset Selection
3. Ridge regression
4. The Lasso
5. Python Lab
6. Summary

Optional and advanced material is collected in the **appendix** at the end of the deck.

Notation in this chapter

Symbol	Meaning
d	Number of predictors in a candidate model (2^p subsets in total)
RSS, TSS	Residual / total sum of squares
$\hat{\sigma}^2$	Estimated noise variance
C_p, AIC	$\frac{1}{n}(\mathrm{RSS} + 2d\hat{\sigma}^2)$ — lighter complexity penalty
BIC	$\frac{1}{n}(\mathrm{RSS} + \log(n)\, d\hat{\sigma}^2)$ — heavier penalty
adj. R^2	R^2 corrected for the model size d
λ	Tuning parameter: penalty strength, chosen by CV
$\lambda \sum_j \beta_j^2$	Ridge (ℓ_2) penalty: shrinks, never exactly zero
$\lambda \sum_j \lvert\beta_j\rvert$	Lasso (ℓ_1) penalty: shrinks and selects

Sources and references

Primary textbook

James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning, with Applications in Python.* Springer.

About these slides

Each method is introduced with motivation, intuition, formal definition, and worked example. Slide titles are descriptive.

Learning objectives for this chapter

By the end of this chapter you will be able to:

- **Apply** best-subset and stepwise selection; **choose** among candidate models with C_p, AIC, BIC, adjusted R^2, or cross-validation.
- **Fit** ridge and lasso regression and **interpret** their coefficient paths.
- **Articulate** the geometric reason the lasso produces sparsity while ridge does not.
- **Tune** λ by cross-validation and **avoid** the leakage that comes from standardising outside the CV loop.

How to use this deck — self-study and revision

Attempt each exercise *before* its solution; run `chapter_06_lab.ipynb` alongside the lab section. Revise from the back — key formulas, vocabulary, self-check (each question names its page) — then redo the exercises; the appendix is fully solved.

Roadmap of this chapter

Select, then shrink, then select *by* shrinking

1. Why plain OLS needs help: variance, interpretability, collinearity.
2. **Subset selection**: best subset, forward and backward stepwise — and how to choose the size with C_p, AIC, BIC or adjusted R^2.
3. **Ridge** (ℓ_2): shrink every coefficient, keep every predictor.
4. **Lasso** (ℓ_1): shrink *and* select, so some coefficients are exactly zero.
5. Which to reach for when, and how to tune λ honestly.

Cross-validation (Ch. 5) is used throughout to choose tuning parameters honestly.

Outline

1. Why this chapter matters
2. Subset Selection
3. Ridge regression
4. The Lasso
5. Python Lab
6. Summary

Three pressures that push us beyond OLS

Plain ordinary least squares is excellent in the classical regime ($n \gg p$, predictors weakly correlated). In modern applications three pressures push us further.

The three motivations

1. **Prediction accuracy.** OLS is unbiased but can have very high variance, especially when $p \approx n$.
2. **Interpretability.** With many predictors, only a few usually matter. We want a model that highlights them.
3. **Collinearity.** When predictors move together, $\mathrm{X}^\top \mathrm{X}$ is near-singular and the OLS coefficients swing wildly with every data refresh.

Two families of solutions

Subset selection

Pick a subset of predictors, then fit OLS on it. Discrete and combinatorial.

Best subset, forward, backward stepwise.

Shrinkage

Keep all predictors but penalise the size of their coefficients. Continuous, tuned by one λ.

Ridge, lasso, elastic net.

Takeaway

Both answer *which predictors should carry weight?* — one by searching, the other by taxing.

Live demo — §1 of the notebook

Switch to `chapter_06_lab.ipynb` §1 and run the loading cell: the Hitters data (263 players, 19 predictors after dummy coding, heavily collinear career totals) is the testbed for both families today.

Outline

1 Why this chapter matters
2 Subset Selection
3 Ridge regression
4 The Lasso
5 Python Lab
6 Summary

Best subset selection: idea

The most direct way to find a small, interpretable linear model.

Algorithm

1. For each $k = 0, 1, \ldots, p$, find the model with k predictors that minimises RSS over all $\binom{p}{k}$ possibilities.
2. Among the $p + 1$ "best per-size" models, choose by cross-validation, C_p, AIC, BIC, or adjusted R^2.

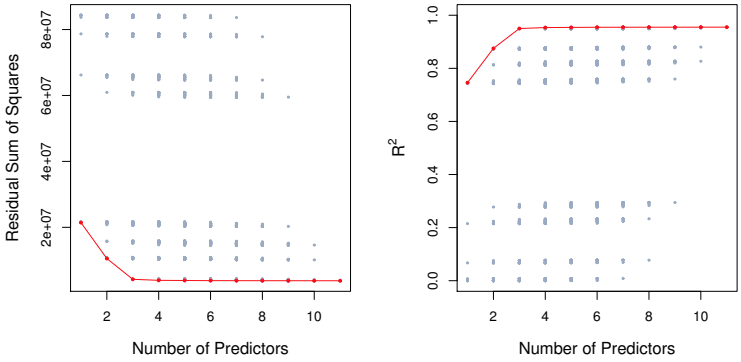
Computational cost

Searching $\binom{p}{k}$ subsets at each k is feasible only for $p \lesssim 30$. Beyond that we use stepwise heuristics.

Live demo — §2 of the notebook

Switch to `chapter_06_lab.ipynb` §2: best subset capped at $k \leq 5$ still fits 16 663 models.

Best subset on the Credit data



RSS and R^2 as k varies: both improve monotonically with more predictors, so they cannot by themselves pick k — that is what C_p, BIC and adjusted R^2 (next) are for.

Source: Figure 6.1 — ISLP, James et al. (2023).

Forward stepwise selection

A greedy alternative when best subset is too expensive.

Algorithm

1. Start with the null model M_0 (only intercept).
2. For $k = 1, \ldots, p$: add to M_{k-1} the predictor that most reduces RSS. Call the result M_k.
3. Choose among $M_0, \ldots, M_p$ by CV / C_p / BIC.

How to read this

Considers $1 + \sum_k (p - k) = O(p^2)$ models instead of 2^p — scales to large p. Greedy: not guaranteed to find the best subset.

Backward and hybrid stepwise

Backward

Start with the full model and drop the least useful predictor at each step. Requires $n > p$.

Hybrid

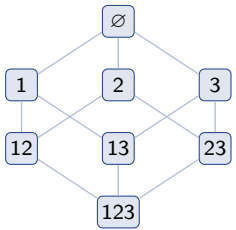
Alternate forward and backward steps. Recovers some of the flexibility of best subset at greedy cost.

All three are heuristics

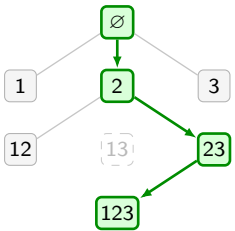
Forward, backward, and hybrid can disagree on the same data. Always cross-validate the chosen model.

Best subset vs. forward stepwise: two search strategies

Best subset: evaluate *all* 2^p



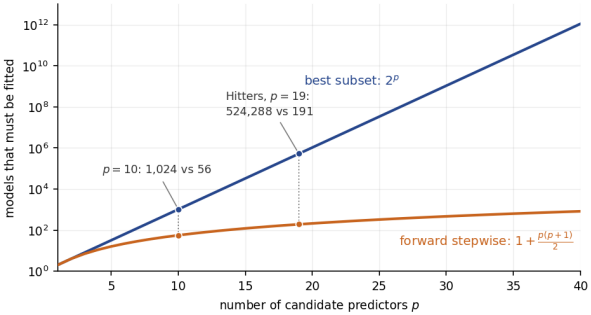
Forward: follow one greedy path



Takeaway

Best subset scores the *whole* lattice (2^p models); forward keeps only the winner at each level (the bold subsets on the arrowed path) and never even fits the dashed subset. At $p = 20$: $2^{20} \approx 10^6$ vs. 211 models.

How fast the search explodes



Takeaway

The lattice, counted. On Hitters ($p = 19$) best subset means 524 288 fits against forward stepwise's 191 — and the log scale is doing the arguing: by $p = 40$ exhaustive search needs a *trillion* fits. This is why the lab caps best subset at $k \leq 5$.

Choosing k: four penalised criteria

Training RSS always decreases with k, so we need penalised criteria that approximate test error.

The four standard criteria

$$
\begin{aligned}
C_p &= \tfrac{1}{n}\big(\mathrm{RSS} + 2d\hat{\sigma}^2\big), \\
\mathrm{AIC} &\propto C_p, \\
\mathrm{BIC} &= \tfrac{1}{n}\big(\mathrm{RSS} + \log(n)\, d\hat{\sigma}^2\big), \\
\text{adj } R^2 &= 1 - \tfrac{\mathrm{RSS}/(n-d-1)}{\mathrm{TSS}/(n-1)}.
\end{aligned}
$$

Takeaway

All four share one shape — *training fit* + *complexity penalty*; they differ only in how steeply each extra predictor is taxed.

Reading the criteria symbol-by-symbol

How to read this

Symbol	Meaning
RSS	Residual sum of squares (training fit)
d	Number of predictors in the model
$\hat{\sigma}^2$	Estimated noise variance
n	Sample size
TSS	Total sum of squares

Practical guidance

BIC penalises more heavily for large $n \Rightarrow$ smaller models. Use BIC when parsimony matters most. Cross-validation is more robust when distributional assumptions are uncertain.

Exercise 6.2 — Why RSS and R^2 Cannot Pick the Size [Concept]

Exercise (≈5 min) — try it before the solution

1. Explain why training RSS and training R^2 can never be used to choose the *number* of predictors d.
2. What quantity do C_p, AIC, and BIC try to estimate that training RSS does not?
3. State qualitatively how C_p, BIC, and adjusted R^2 modify the raw training fit, and which of them most favours small models.

Solution — Exercise 6.2

Solution

(1) Monotonicity. Adding a predictor to a nested least-squares model can never increase RSS (the extra coefficient can always be set to zero), so RSS is *non-increasing* in d and $R^2 = 1 - \mathrm{RSS}/\mathrm{TSS}$ is *non-decreasing*. Both are therefore optimised by the largest model, giving no guidance on size and rewarding overfitting.
(2) What the criteria estimate. They estimate the *test* error (expected prediction error on new data), not the training error. Each adds a penalty that grows with d to counteract the optimism of the training fit.
(3) How they adjust the fit.

- $C_p = \frac{1}{n}(\mathrm{RSS} + 2d\hat{\sigma}^2)$: adds a penalty $2d\hat{\sigma}^2$ (AIC is proportional to it).
- $\mathrm{BIC} = \frac{1}{n}(\mathrm{RSS} + \log(n)\, d\hat{\sigma}^2)$: same shape but penalty $\log(n)\, d\hat{\sigma}^2$. Since $\log n > 2$ for $n > 7$, BIC penalises extra predictors more heavily and therefore *most favours small (parsimonious) models*.
- Adjusted $R^2 = 1 - \frac{\mathrm{RSS}/(n-d-1)}{\mathrm{TSS}/(n-1)}$: divides RSS by its degrees of freedom, so it can *decrease* when a useless predictor is added.

We pick the model that minimises C_p/AIC/BIC or maximises adjusted R^2.

Common mistake: comparing models of different sizes by training RSS or R^2 — the biggest model always wins by construction; that is exactly the problem these criteria solve.

Exercise 6.3 — Computing C_p and BIC [Math]

Exercise (≈5 min) — try it before the solution

With $n = 100$ observations and $\hat{\sigma}^2 = 4$, two candidate models give:

	d	RSS
Model A	3	480
Model B	6	450

Compute C_p and BIC for each model and state which model each criterion selects. Use $C_p = \frac{1}{n}(\mathrm{RSS} + 2d\hat{\sigma}^2)$ and $\mathrm{BIC} = \frac{1}{n}(\mathrm{RSS} + \log(n)\, d\hat{\sigma}^2)$.

Solution — Exercise 6.3 (1/3)

Solution

Here $n = 100$, $\hat{\sigma}^2 = 4$, and $\log(100) \approx 4.605$. Both criteria share the shape $\frac{1}{n}(\mathrm{RSS} + \text{price} \times d\hat{\sigma}^2)$; only the per-predictor price differs (2 for C_p vs. $\log n$ for BIC).
C_p (price 2 per predictor).

$$C_p^A = \tfrac{1}{100}\big(480 + 2 \cdot 3 \cdot 4\big) = \tfrac{1}{100}(480 + 24) = 5.04,$$
$$C_p^B = \tfrac{1}{100}\big(450 + 2 \cdot 6 \cdot 4\big) = \tfrac{1}{100}(450 + 48) = 4.98.$$

C_p is smaller for **Model B** ($4.98 < 5.04$), so C_p selects B: B's RSS saving of $480 - 450 = 30$ outweighs its extra penalty of $48 - 24 = 24$.

Solution — Exercise 6.3 (2/3)

Solution

BIC (price $\log(100) \approx 4.605$ per predictor).

$$\mathrm{BIC}^A = \tfrac{1}{100}\big(480 + 4.605 \cdot 3 \cdot 4\big) = \tfrac{1}{100}(480 + 55.3) = 5.353,$$
$$\mathrm{BIC}^B = \tfrac{1}{100}\big(450 + 4.605 \cdot 6 \cdot 4\big) = \tfrac{1}{100}(450 + 110.5) = 5.605.$$

BIC is smaller for **Model A** ($5.353 < 5.605$), so BIC selects A: the same RSS saving of 30 must now beat an extra penalty of $110.5 - 55.3 = 55.2$ — and fails.

Solution — Exercise 6.3 (3/3)

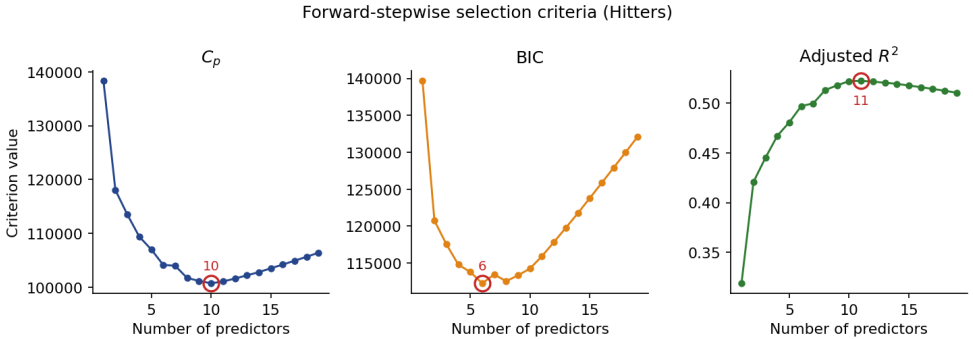
Solution

Interpretation. The two criteria disagree: BIC's heavier $\log(n)$ penalty punishes the three extra predictors of Model B more than they reduce RSS, so BIC prefers the smaller Model A, while the lighter C_p penalty lets Model B's better fit win. When parsimony matters, follow BIC.

Common mistake

Using $\log_{10}(100) = 2$ instead of the natural log $\ln(100) \approx 4.605$ — with base-10 logs the BIC penalty would collapse to C_p's $2d\hat{\sigma}^2$ here.

Choosing the model size: C_p, BIC and adjusted R^2



Takeaway

Each criterion trades goodness-of-fit against complexity; BIC's heavier penalty picks the smallest model, C_p and adjusted R^2 a slightly larger one.

Outline

1. Why this chapter matters
2. Subset Selection
3. Ridge regression
4. The Lasso
5. Python Lab
6. Summary

Why ridge: numerical stability via a penalty

Subset selection is discrete and combinatorial. A smoother alternative: keep all predictors but penalise large coefficients.

Intuition

The penalty pulls $\hat{\beta}$ toward zero, especially when the data don't strongly constrain it. We trade a small upward bias for a large reduction in variance.

How to read this

Two payoffs at once: the penalty (i) *stabilises* the fit when predictors are collinear — where plain OLS returns huge, sign-flipping coefficients that move with every data refresh — and (ii) *lowers variance* by refusing to chase noise with large coefficients.

The ridge regression objective

Objective

$$\hat{\beta}^{\mathrm{R}} = \arg\min_{\beta} \Bigg\{ \underbrace{\sum_{i=1}^{n} (y_i - \beta_0 - X_i^{\top}\beta)^2}_{\mathrm{RSS}} + \underbrace{\lambda \sum_{j=1}^{p} \beta_j^2}_{\ell_2 \text{ penalty}} \Bigg\}.$$

Takeaway

Minimise training error *plus* a size tax on the coefficients; the tuning parameter λ sets how hard we pull them toward zero.

History: proposed by Hoerl & Kennard (1970) to stabilise regressions with correlated predictors in industrial chemistry; the same idea appears in maths as Tikhonov regularisation.

Reading the ridge objective

How to read this

Term	Role
RSS	Training fit: pulls coefficients toward the OLS solution.
$\lambda \sum \beta_j^2$	Penalty: pulls coefficients toward zero.
λ	Tuning parameter that controls the trade-off.

- $\lambda = 0$ recovers ordinary least squares.
- $\lambda \to \infty$ drives every $\hat{\beta}^{\mathrm{R}} \to 0$.
- Closed form: $\hat{\beta}^{\mathrm{R}} = (\mathrm{X}^\top \mathrm{X} + \lambda I)^{-1} \mathrm{X}^\top \mathrm{y}$.

Live demo — §3 of the notebook

Switch to `chapter_06_lab.ipynb` §3 and run the ridge cell: CV picks $\alpha \approx 2.8$ and all 19 coefficients stay nonzero — shrunk, never zeroed.

Three ways to think about ridge

Numerical stability

Adding λI stabilises the inverse when $\mathrm{X}^\top \mathrm{X}$ is near-singular (collinearity).

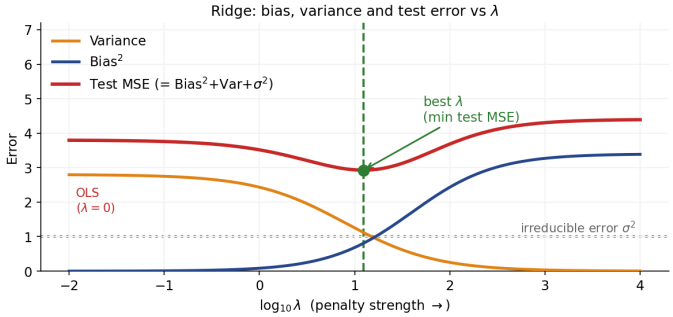
Bias-variance trade-off

Trades a small upward bias for a large reduction in variance. Especially helpful when p is large relative to n.

Standardise first!

The penalty is not scale-invariant. Standardise predictors before fitting, otherwise ridge will preferentially shrink variables with larger units.

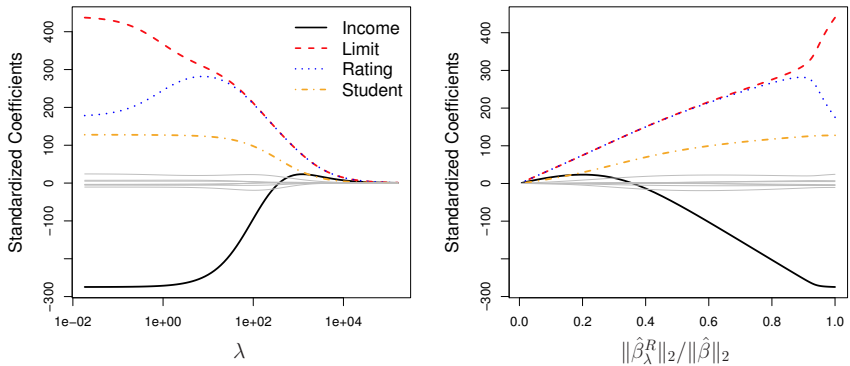
Ridge's bias-variance trade-off, visualised



Takeaway

As λ grows, variance falls but squared bias rises. Test error is their sum plus the irreducible σ^2, so it bottoms out at an *intermediate* λ — the one cross-validation aims to find.

Ridge: coefficient paths



Each line is one $\hat{\beta}_j$ as λ grows from left to right. Coefficients shrink smoothly toward zero — but never exactly reach zero.

Source: Figure 6.4 — ISLP, James et al. (2023).

Exercise 6.4 — Ridge: λ and Standardisation [Concept]

Exercise (≈5 min) — try it before the solution

1. Describe what happens to the ridge coefficients $\hat{\beta}^{\mathrm{R}}$ as λ increases from 0 to ∞, and to the model's bias and variance.
2. Why must predictors be standardised before ridge regression? What goes wrong if one predictor is measured in metres and another in millimetres?
3. Does ridge ever set a coefficient exactly to zero? What does this imply for interpretability?

Solution — Exercise 6.4 (1/2)

Solution

(1) Effect of λ. At $\lambda = 0$ ridge equals OLS. As λ grows the ℓ_2 penalty pulls all coefficients smoothly toward zero, and as $\lambda \to \infty$ every $\hat{\beta}_j^{\mathrm{R}} \to 0$ (only the intercept survives). Increasing λ *increases bias* (the fit is pulled away from the data) but *decreases variance* (the coefficients become more stable). The best λ, chosen by cross-validation, trades a little bias for a large variance reduction.

(2) Why standardise. The penalty $\lambda \sum_j \beta_j^2$ is *not scale invariant*. A predictor measured in metres has a coefficient $1000\times$ larger than the same predictor in millimetres, so it would be penalised 10^6 times more heavily and shrunk far more aggressively — an arbitrary artefact of units, not of importance. Standardising each predictor to unit variance (typically $\tilde{x}_{ij} = x_{ij}/\hat{\sigma}_j$) puts them on a common footing so the penalty treats them comparably.

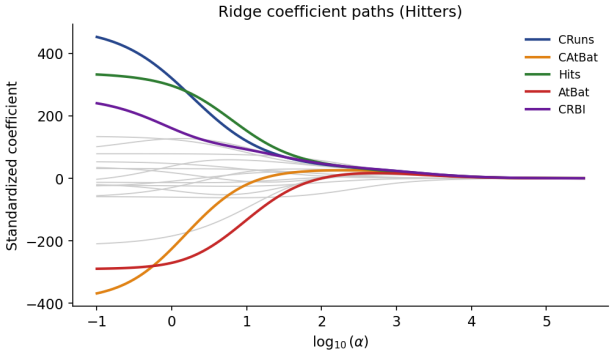
Solution — Exercise 6.4 (2/2)

Solution

(3) No exact zeros. Ridge shrinks coefficients toward zero but essentially never sets them *exactly* to zero (the ℓ_2 penalty is smooth at the origin). So ridge keeps all p predictors; it improves prediction and stability but does *not* select variables — for that, use the lasso.

Common mistake: treating standardisation as cosmetic. For plain OLS it is (predictions unchanged); with a penalty it *changes the model* — ridge on raw and on standardised predictors gives genuinely different fits.

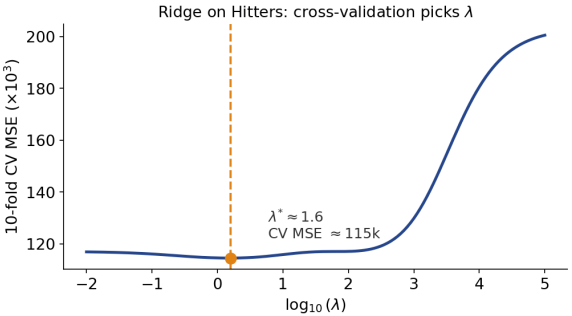
Ridge coefficient paths on Hitters



Takeaway

As λ grows every coefficient shrinks smoothly toward zero, but none is set *exactly* to zero — ridge keeps all predictors.

Choosing λ by cross-validation on Hitters



Takeaway

The 10-fold CV curve on standardised Hitters dips at $\lambda^* \approx 1.6$ (CV MSE $\approx$ 114,000) — mild shrinkage beats OLS, and the error climbs steeply once heavy shrinkage drowns the signal.

Outline

1 Why this chapter matters
2 Subset Selection
3 Ridge regression
4 The Lasso
5 Python Lab
6 Summary

The lasso: change the penalty, change the behaviour

Same idea as ridge but with an ℓ_1 penalty instead of ℓ_2.

Objective

$$\hat{\beta}^{\mathrm{L}} = \arg\min_{\beta} \left\{ \mathrm{RSS} + \lambda \sum_{j=1}^{p} |\beta_j| \right\}.$$

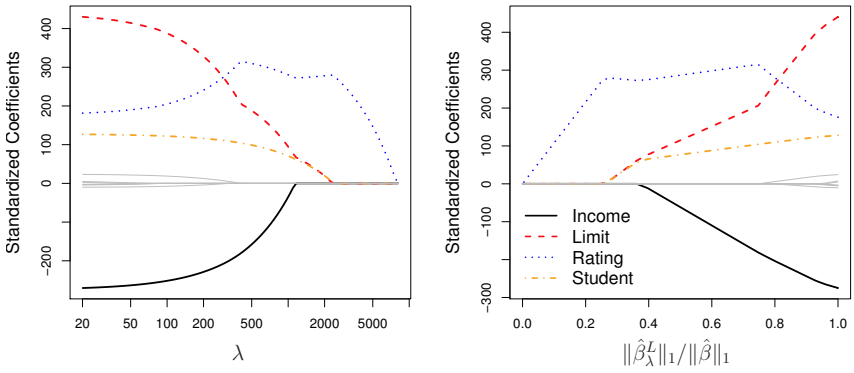
How to read this

The *only* change from ridge is $|\beta_j|$ in place of β_j^2 — and that kink at the origin is exactly what lets coefficients reach zero.

The crucial property

For λ large enough, some $\hat{\beta}_j^{\mathrm{L}} = 0$ *exactly*. The lasso *performs variable selection* automatically as a side effect of fitting.

Lasso: coefficient paths



History: the lasso is Robert Tibshirani's (1996) — one of the authors of our textbook. The name is an acronym: least absolute shrinkage and selection operator. As λ grows, coefficients hit zero one by one. The lasso delivers a continuum of sparse models indexed by λ.

Source: Figure 6.6 — ISLP, James et al. (2023).

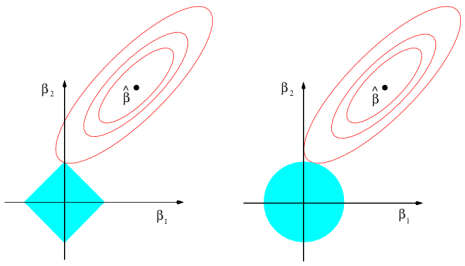
Why does lasso produce zeros? — the geometry

The unit balls

- Ridge constrains $\sum \beta_j^2 \leq t$ — a smooth ball.
- Lasso constrains $\sum |\beta_j| \leq t$ — a polytope with corners on the axes.

Takeaway

The level sets of the squared-error loss are likely to first touch the lasso ball at a corner — a corner has some $\beta_j = 0$.



Source: Figure 6.7 — ISLP, James et al. (2023).

Choose λ by cross-validation

Both ridge and lasso introduce a single tuning parameter.

The procedure

1. Run k-fold CV over a logarithmic grid of λ values (say, 50 from 10^{-3} to 10^{3}).
2. Pick the λ that minimises CV error, or apply the *one-standard-error rule*: the largest λ within one standard error of the minimum.
3. Re-fit on the full data at that λ.

How to read this

In Python: `sklearn.linear_model.RidgeCV` and `LassoCV` perform the grid search internally.

Takeaway

It is λ — not the coefficient list — that you defend.

Ridge vs. lasso vs. elastic net: when each wins

Ridge	Lasso	Elastic net
Many small effects, none dominant. Predictors all somewhat useful.	Truth is sparse: a few predictors carry the signal, the rest are noise.	$\alpha\|\beta\|_1 + (1-\alpha)\|\beta\|_2^2$. Useful for sparse + correlated predictors.

Default workflow

Try lasso first. If you see correlated predictors splitting their estimated effects, fall back to elastic net or ridge. And remember that the lasso's chosen *set* is itself unstable under correlation — re-run it over bootstrap samples and believe only the variables that survive most of them.

Mini-check: ridge vs. lasso

Quick self-assessment

For each scenario, which method is more appropriate?

1. Many predictors, all expected to have small effects.
2. Truth is sparse: only a handful of predictors matter.
3. Severe multicollinearity, want a stable fit.
4. Want automatic variable selection.

Answers

1. Ridge. 2. Lasso. 3. Ridge. 4. Lasso (or elastic net).

Live demo — §4 of the notebook

Switch to `chapter_06_lab.ipynb` §4 and run the lasso cell: CV picks $\alpha \approx 2.3$ and zeroes 5 of the 19 predictors; the paths cell then shows them hitting zero one by one.

Exercise 6.5 — Lasso Sparsity: ℓ_1 vs. ℓ_2 [Concept/Math]

Exercise (≈5 min) — try it before the solution

1. Write the constrained (budget) form of ridge and lasso and describe the shape of each constraint region in two dimensions.
2. Use that geometry to explain why the lasso produces exact zeros while ridge does not.
3. *Orthonormal design.* When $\mathsf{X}^\top \mathsf{X} = I$, the lasso has the closed-form soft-threshold solution $\hat{\beta}_j^{\mathrm{L}} = \mathrm{sign}(\hat{\beta}_j^{\mathrm{OLS}})\big(|\hat{\beta}_j^{\mathrm{OLS}}| - \lambda/2\big)_+$. If $\hat{\beta}_j^{\mathrm{OLS}} = 1.5$ and $\lambda = 4$, compute $\hat{\beta}_j^{\mathrm{L}}$.

Solution — Exercise 6.5 (1/2)

Solution

(1) Constraint regions. Both methods can be written as minimise RSS subject to a budget:

$$\text{ridge: } \sum_j \beta_j^2 \le t, \qquad \text{lasso: } \sum_j |\beta_j| \le t.$$

In two dimensions the ridge region is a *disc* (smooth, rounded), while the lasso region is a *diamond* (a square rotated 45°) with sharp corners lying on the coordinate axes.

(2) Why lasso gives zeros. The elliptical contours of the RSS expand until they first touch the constraint region; the solution is that contact point. The lasso diamond has corners *on the axes*, where one coordinate is exactly zero, and contours are very likely to hit a corner first — yielding $\hat{\beta}_j = 0$. The ridge disc has no corners, so the contact point almost never lands exactly on an axis; ridge shrinks but does not zero out. This is why the ℓ_1 penalty performs variable selection and the ℓ_2 penalty does not.

Solution — Exercise 6.5 (2/2)

Solution

(3) Soft-thresholding. With $\hat{\beta}_j^{\mathrm{OLS}} = 1.5$ and $\lambda = 4$:

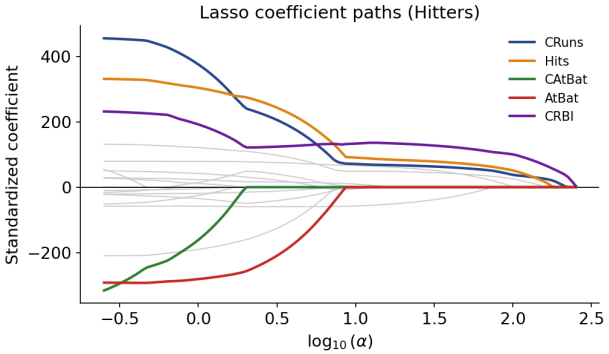
$$|\hat{\beta}_j^{\mathrm{OLS}}| - \lambda/2 = 1.5 - 2.0 = -0.5 < 0,$$

so $(\cdot)_+ = 0$ and $\hat{\beta}_j^{\mathrm{L}} = 0$: the predictor is dropped. Had $\lambda = 2$ instead, we would get $\mathrm{sign}(1.5)(1.5 - 1.0) = +0.5$ — shrunk toward zero but retained. This shows algebraically how the lasso zeros out coefficients once the penalty exceeds twice the OLS magnitude, while the ridge solution $\hat{\beta}_j/(1+\lambda)$ would only shrink, never reaching zero.

Common mistake

Thresholding at λ instead of $\lambda/2$. Here both give zero, but at $\lambda = 2$ the wrong threshold yields $1.5 - 2 < 0 \Rightarrow 0$, killing a coefficient that actually survives as $1.5 - 1.0 = 0.5$.

Lasso coefficient paths on Hitters



Takeaway

Unlike ridge, the lasso drives coefficients to *exactly* zero as λ grows — it performs automatic variable selection.

Outline

1. Why this chapter matters
2. Subset Selection
3. Ridge regression
4. The Lasso
5. Python Lab
6. Summary

Python lab — run it live in the notebook

Companion notebook: `chapter_06_lab.ipynb`

The code in this section is meant to be **demonstrated live**. Open the companion Jupyter notebook and run it cell by cell, section by section:

- §2, *Best-subset and forward stepwise selection* — on the Hitters data loaded in §1;
- §3, *Ridge regression*, and §4, *Lasso* — each with its coefficient-path subsection;
- §5, *How many models did we skip?* — the search space, counted.

Data loads via the `ISLP` package or the bundled CSV files; §6, *Exercises*, closes the notebook.

Best subset and stepwise (manual)

```
import statsmodels.api as sm, itertools # OLS fits + subset lists
from ISLP import load_data
Hitters = load_data("Hitters").dropna() # NA salaries out: n=263
y = Hitters["Salary"] # response we predict
preds = [c for c in Hitters.columns if c not in ("Salary",)] # 19 preds

best = {}
for k in range(1, 6): # model sizes k = 1..5
    best_rss = None
    for combo in itertools.combinations(preds, k): # all C(p,k) subsets
        X = sm.add_constant(Hitters[list(combo)]) # subset + intercept
        rss = sm.OLS(y, X).fit().ssr # training RSS of subset
        if best_rss is None or rss < best_rss: # lower RSS wins
            best_rss, best_combo = rss, combo # keep best model per size
    best[k] = (best_combo, best_rss)
# RSS only ranks equal-size models; Cp/BIC/CV then pick k. k<=5 with
# p=19 is already 16,663 OLS fits -- and 2^19 is the full search.
```

Ridge regression

```
from sklearn.linear_model import Ridge # L2 penalty: sum beta_j^2
from sklearn.model_selection import GridSearchCV # sweeps the grid
from sklearn.pipeline import make_pipeline # scaler + model as one unit
from sklearn.preprocessing import StandardScaler # z-score each column
import numpy as np

X, y = Hitters[preds], Hitters["Salary"].values # n=263, p=19
# Scale INSIDE the search: re-fit per fold, else CV flatters itself.
ridge = GridSearchCV(make_pipeline(StandardScaler(), Ridge()), # 50 alphas
                     {"ridge__alpha": np.logspace(-2, 4, 50)}, cv=10,
                     scoring="neg_mean_squared_error").fit(X, y) # not R^2
print("best alpha:", ridge.best_params_["ridge__alpha"]) # 2.81 here
print(dict(zip(preds, ridge.best_estimator_[-1].coef_))) # all stay nonzero
```

How to read this

The penalty is not scale-invariant, so standardising is mandatory.

The lasso

```
from sklearn.linear_model import Lasso # L1 penalty: sum |beta_j|

# same discipline; 50 alphas, 1e-3 to 10: near-OLS -> heavily sparse
lasso = GridSearchCV(make_pipeline(StandardScaler(), Lasso(max_iter=10000)),
                     {"lasso__alpha": np.logspace(-3, 1, 50)}, cv=10,
                     scoring="neg_mean_squared_error").fit(X, y)
print("best alpha:", lasso.best_params_["lasso__alpha"]) # CV winner
coefs = lasso.best_estimator_[-1].coef_ # [-1] = the fitted Lasso step
print([(p, c) for p, c in zip(preds, coefs) if c != 0]) # only the survivors
```

How to read this

Only the penalty changed: $\sum |\beta_j|$ replaces $\sum \beta_j^2$. The ℓ_1 ball has *corners* on the axes, so the CV-chosen α sets weak coefficients to *exactly* zero instead of merely small, so the printed list is shorter than `preds`; ridge on the same columns leaves all 19 nonzero. Standardise for the same reason — the penalty compares raw coefficients.

Exercise 6.6 — CV Ridge and Lasso on Hitters [Python]

Exercise (≈5 min) — try it before the solution

On the `Hitters` data (predict `Salary`), (i) fit lasso with a cross-validated λ on standardised predictors, (ii) report the selected λ, and (iii) list which features the lasso keeps versus drops. Contrast with ridge. What do the results tell you?

Run this live

The worked solution sits in `chapter_06_lab.ipynb` under *Lecture exercises — worked Python solutions*: the cell headed *Exercise 6.6 — CV Ridge and Lasso on Hitters [Python]*.

Solution — Exercise 6.6 (1/3)

```
import numpy as np, pandas as pd
from sklearn.linear_model import Lasso, Ridge # L1 and L2 penalty
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler # z-score columns
# path as seen from Chapters/, where you run the notebook
df = pd.read_csv("../ALL CSV FILES - 2nd Edition/Hitters.csv").dropna()
y = df["Salary"].values # response, n=263 rows
X = pd.get_dummies(df.drop(columns=["Salary"]), drop_first=True) # p=19
preds = X.columns # scale inside the CV, not here
lasso = GridSearchCV(make_pipeline(StandardScaler(), Lasso(max_iter=10000)),
                     {"lasso__alpha": np.logspace(-3, 2, 50)}, cv=10,
                     scoring="neg_mean_squared_error").fit(X, y)
print("lasso alpha:", round(lasso.best_params_["lasso__alpha"], 3))
coefs = lasso.best_estimator_[-1].coef_ # [-1] = the fitted Lasso
kept = [p for p, c in zip(preds, coefs) if abs(c) > 1e-8] # 14 of 19
dropped = [p for p, c in zip(preds, coefs) if abs(c) <= 1e-8] # 5 zeros
print("kept :", kept, "\ndropped:", dropped) # retained vs zeroed exactly
```

Solution — Exercise 6.6 (2/3)

```
ridge = GridSearchCV(make_pipeline(StandardScaler(), Ridge()), # scale in CV
                     {"ridge__alpha": np.logspace(-2, 4, 50)}, cv=10,
                     scoring="neg_mean_squared_error").fit(X, y)
rc = ridge.best_estimator_[-1].coef_ # [-1] = the fitted Ridge
print("ridge alpha:", round(ridge.best_params_["ridge__alpha"], 3),
      "-- nonzero coefs:", int(np.sum(np.abs(rc) > 1e-8))) # all 19
```

Solution — Exercise 6.6 (3/3)

Solution — expected result and interpretation

The lasso selects a moderate λ and *drives several coefficients to exactly zero* — typically keeping strong career and recent-performance predictors (`CRBI`, `Hits`, `Walks`, `PutOuts`, `DivisionW`) and dropping the weak, redundant ones. Ridge keeps *all* predictors nonzero, only shrinking them: lasso buys a sparse, interpretable model by doing variable selection; ridge stabilises the fit and keeps every predictor.

Common mistake

Comparing the printed `alpha` with a textbook λ — `scikit-learn` scales the lasso penalty by $1/(2n)$, so the numbers differ across formulations even when the fitted models agree.

Extended Exercise 6.3 — CV Ridge and Lasso Test MSE on Hitters [Python]

Extended exercise (≈15 min) — try it before the solution

On the `Hitters` data (predict `Salary`; drop missing salaries; dummy-encode `League`, `Division`, `NewLeague`):

1. Split into train/test, standardise predictors *using the training set only*, and cross-validate to choose λ for both ridge and lasso.
2. Report each chosen λ, the *test* MSE, and how many coefficients are non-zero.
3. List the predictors the lasso drops and contrast ridge vs. lasso.

Run this live

The worked solution sits in `chapter_06_lab.ipynb` under *Lecture exercises — worked Python solutions*: the cell headed *Extended Exercise 6.3 — CV Ridge and Lasso Test MSE on Hitters [Python]*.

Solution — Extended Exercise 6.3 (1/4)

```
import numpy as np, pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV, LassoCV # built-in CV
from sklearn.metrics import mean_squared_error # test score

D = "../ALL CSV FILES - 2nd Edition/" # path from Chapters/
df = pd.read_csv(D+"Hitters.csv").dropna(subset=["Salary"]) # n=263
y = df["Salary"].values # response: the 263 salaries
X = pd.get_dummies(df.drop(columns=["Salary"]),
                   drop_first=True).astype(float) # -> 19 predictors
Xtr, Xte, ytr, yte = train_test_split(X.values, y,
                      test_size=0.33, random_state=1) # 2/3-1/3 split
# The penalty acts on raw coefficients, so units decide who gets shrunk:
# standardise, but learn the means and sds from training rows only.
sc = StandardScaler().fit(Xtr) # fit on TRAIN only: no leakage
Xtr, Xte = sc.transform(Xtr), sc.transform(Xte) # apply to both sets
```

Solution — Extended Exercise 6.3 (2/4)

```
ridge = RidgeCV(alphas=np.logspace(-2, 4, 50)).fit(Xtr, ytr) # CV on Xtr
lasso = LassoCV(cv=10, max_iter=100000,
                random_state=0).fit(Xtr, ytr) # 10-fold CV for lambda

for name, m in [("ridge", ridge), ("lasso", lasso)]: # same report
    mse = mean_squared_error(yte, m.predict(Xte)) # MSE on held-out
    nz = int(np.sum(np.abs(m.coef_) > 1e-8)) # 19 ridge vs 16 lasso
    print(name, "lambda=", round(m.alpha_, 3),
          "testMSE=", round(mse, 0), "nonzero=", nz)

dropped = [c for c, w in zip(X.columns, lasso.coef_)
           if abs(w) <= 1e-8] # Years, CRBI, Errors here
print("lasso drops:", dropped)
```

How to read this

λ comes from CV on the training rows; the MSE is on the test third.

Solution — Extended Exercise 6.3 (3/4)

Solution — expected result and interpretation

With $n = 263$, $p = 19$ and this split you should see roughly:

- **Ridge**: $\lambda \approx 0.1$, test MSE $\approx 1.2 \times 10^5$, *all* 19 *coefficients non-zero*.

Solution — Extended Exercise 6.3 (4/4)

Solution — expected result and interpretation

- **Lasso**: $\lambda \approx 0.8$, test MSE $\approx 1.2 \times 10^5$, about 16 non-zero — it drops `Years`, `CRBI`, `Errors` (weak or redundant given the correlated career totals).

Contrast. Both attain very similar test error, but the lasso reaches it with a *sparser, more interpretable* model (automatic variable selection), whereas ridge keeps and merely shrinks all predictors. Exact numbers shift with the split/seed; the qualitative message — lasso selects, ridge keeps all — is stable.

Common mistake

Reading the dropped variables as "irrelevant to salary". `Years` and `CRBI` are highly correlated with the kept career totals — the lasso drops *redundant* predictors, and a different split can swap which of a correlated group survives.

Outline

1. Why this chapter matters
2. Subset Selection
3. Ridge regression
4. The Lasso
5. Python Lab
6. Summary

Chapter 6 in one slide

What we accomplished

Two families of methods that turn the linear model into a workhorse for wide, collinear and sparse problems.

- Subset selection (best, forward, backward), scored by C_p, AIC, BIC or adjusted R^2.
- Shrinkage (ridge, lasso, elastic net).
- The geometry that makes the lasso — and only the lasso — set coefficients to exactly zero.
- Honest tuning via cross-validation throughout.

Key formulas at a glance

C_p	$C_p = \frac{1}{n}\big(\mathrm{RSS} + 2d\hat{\sigma}^2\big)$ (lower is better).
AIC / BIC	$\mathrm{AIC} \propto \mathrm{RSS} + 2d\hat{\sigma}^2$; $\mathrm{BIC} = \frac{1}{n}\big(\mathrm{RSS} + \log(n)\, d\, \hat{\sigma}^2\big)$.
Adjusted R^2	$1 - \dfrac{\mathrm{RSS}/(n-d-1)}{\mathrm{TSS}/(n-1)}$.
Ridge (ℓ_2)	$\min_\beta \ \mathrm{RSS} + \lambda \sum_{j=1}^{p} \beta_j^2$ (shrinks, keeps all predictors).
closed form	$\hat{\beta}^{\mathrm{R}} = (\mathsf{X}^\top \mathsf{X} + \lambda I)^{-1} \mathsf{X}^\top \mathsf{y}$ (β_0 unpenalised).
Lasso (ℓ_1)	$\min_\beta \ \mathrm{RSS} + \lambda \sum_{j=1}^{p} \lvert\beta_j\rvert$ (shrinks and selects: some $\hat{\beta}_j = 0$).
soft-threshold	$\hat{\beta}_j^{\mathrm{L}} = \mathrm{sign}(\hat{\beta}_j^{\mathrm{OLS}})\big(\lvert\hat{\beta}_j^{\mathrm{OLS}}\rvert - \lambda/2\big)_+$ (orthonormal $\mathsf{X}^\top \mathsf{X} = I$ only; no general closed form).
Tuning	choose d or λ by cross-validation.

Vocabulary checklist

Term	Meaning
Best subset	Search all $\binom{p}{k}$ models for each k
Forward / backward	Greedy add / drop one predictor at a time
C_p / AIC / BIC	Penalised RSS criteria for model size
Adjusted R^2	R^2 penalised for the number of predictors
Ridge (ℓ_2)	Penalty $\lambda \sum \beta_j^2$
Lasso (ℓ_1)	Penalty $\lambda \sum \lvert\beta_j\rvert$
Elastic net	Mix of ℓ_1 and ℓ_2 penalties
Tuning parameter λ	Strength of the penalty
One-standard-error rule	Largest λ within one SE of the CV minimum
Sparsity	Many estimated β_j exactly zero

Decision rules of thumb

- Always standardise predictors before ridge or lasso.
- Choose λ by 10-fold CV; consider the one-standard-error rule for parsimony.
- Use BIC for parsimonious model selection; AIC / C_p for predictive accuracy.
- With strongly collinear predictors prefer ridge (or elastic net): the lasso's chosen *set* is the unstable part.
- Re-fit OLS on the lasso-selected variables ("relaxed lasso") if you want unbiased coefficients with the sparse support.

Common pitfalls

Don't

1. Apply lasso / ridge without standardising.
2. Treat post-selection p-values as valid (module A7).
3. Use AIC / BIC once p approaches n — they assume $n \gg p$.
4. Tune λ on the same data you evaluate on.
5. Standardise on the full data before cross-validating — the scaler must be re-fit inside every fold.
6. Read the lasso's dropped variables as "irrelevant" when they are merely redundant with a kept one.

Connections to the rest of the course

- Ch. 3: linear regression with no penalty.
- Ch. 5: how to choose λ honestly via CV.
- Ch. 7: GAMs are an additive ridge-like extension.
- Ch. 8: trees and boosting tame many correlated predictors in a quite different way.
- Module A9: deep nets use weight decay (ridge in disguise).
- Module A7: post-selection inference and the lasso.

Self-check questions

Each question names the slide that answers it — a wrong answer tells you where to read.

1. Why does ridge regression always have a unique solution? (p. 30)
2. Why can lasso set coefficients to exactly zero, but ridge cannot? Sketch the unit balls. (p. 42)
3. For two perfectly correlated predictors, what does ridge do? What does lasso do? What does elastic net do? (p. 44)
4. Compute the ridge estimate when $X^\top X = I$ — a clean special case. (appendix, p. 84)
5. Why must the scaler be re-fit inside every cross-validation fold rather than once on the full data? (p. 53)
6. Outline the one-standard-error rule. (p. 43)

Seven things to remember

1. Subset selection: clean conceptually but combinatorial.
2. Stepwise selection: greedy and fast.
3. C_p, AIC, BIC and adjusted R^2 all read "training fit + complexity penalty"; CV needs no such assumption.
4. Ridge: ℓ_2 penalty $\rightarrow$ smooth shrinkage, no zeros.
5. Lasso: ℓ_1 penalty $\rightarrow$ sparse solutions.
6. Always standardise before shrinkage — inside the CV loop.
7. Choose λ by cross-validation.

What's next

Chapter 7: *Moving Beyond Linearity*.

- Polynomial and step functions.
- Regression splines and smoothing splines.
- Local regression.
- Generalised additive models.

Appendix — what is in here, and why

Nothing in the appendix is needed to follow the chapter: each slide is a formal derivation, a longer worked exercise, or a side topic. Read it when you want the full story, or when a later chapter sends you back here.

Contents of the appendix

- **Extended Exercise 6.1 — all three criteria across model sizes** — the full table, three solution slides. Exercise 6.3 in the main deck is the exam-shaped short version.
- **The constraint geometry, drawn a second time** — a duplicate of the diamond-vs-circle picture that is already in the main deck.
- **Counting the models (Exercise 6.1)** — how many subsets best subset and forward stepwise actually fit, 2^p against $1 + p(p+1)/2$. The lattice picture in the main deck makes the same point in one slide.
- **Ridge and lasso under an orthonormal design (Extended Exercise 6.2)** — the closed forms that explain *why* the lasso thresholds coefficients to zero and ridge only shrinks them. The cleanest proof in the chapter — and the hardest algebra.

Extended Exercise 6.1 — C_p, BIC and Adjusted R^2 Across Sizes [Math]

Extended exercise (≈15 min) — try it before the solution

Best-subset selection on a problem with $n = 100$ and $p = 4$ returns the best model of each size, with residual sums of squares (and $\hat{\sigma}^2 = 5$ estimated from the full model):

d	0	1	2	3	4
RSS	900	500	340	322	314

1. Compute C_p, BIC and adjusted R^2 for each size, using $C_p = \frac{1}{n}(\mathrm{RSS} + 2d\hat{\sigma}^2)$, $\mathrm{BIC} = \frac{1}{n}(\mathrm{RSS} + \log(n)d\hat{\sigma}^2)$ and $\mathrm{adj}\, R^2 = 1 - \frac{\mathrm{RSS}/(n-d-1)}{\mathrm{TSS}/(n-1)}$ with $\mathrm{TSS} = \mathrm{RSS}_{d=0}$.
2. State the winner under each criterion and explain any disagreement.
3. How many models did best subset examine, and how many would forward stepwise fit?

Solution — Extended Exercise 6.1 (1/3)

Solution

With $n = 100$, $\hat{\sigma}^2 = 5$ and $\log(100) \approx 4.605$, the per-parameter penalties are $2\hat{\sigma}^2 = 10$ (C_p) and $\log(n)\hat{\sigma}^2 \approx 23.03$ (BIC). Dividing by $n = 100$:

d	0	1	2	3	4
C_p	9.00	5.10	3.60	3.52	3.54
BIC	9.00	5.23	3.86	3.91	4.06

For example $C_p(3) = \frac{1}{100}(322 + 2 \cdot 3 \cdot 5) = \frac{352}{100} = 3.52$ and $\mathrm{BIC}(2) = \frac{1}{100}(340 + 4.605 \cdot 2 \cdot 5) = \frac{386.05}{100} = 3.86$.

Solution — Extended Exercise 6.1 (2/3)

Solution

Adjusted R^2 with $\mathrm{TSS} = 900$ and $\mathrm{TSS}/(n-1) = 900/99 = 9.09$:

d	0	1	2	3	4
$\mathrm{RSS}/(n-d-1)$	9.09	5.10	3.51	3.35	3.31
$\mathrm{adj}\, R^2$	0.00	0.44	0.61	0.63	0.636

e.g. at $d = 4$: $1 - \frac{314/95}{900/99} = 1 - \frac{3.305}{9.091} = 0.636$.

Winners. $C_p \Rightarrow d = 3$; $\mathrm{BIC} \Rightarrow d = 2$; $\mathrm{adj}\, R^2 \Rightarrow d = 4$.

Common mistake

Dividing by $n - d$ instead of $n - d - 1$ in the numerator — the extra -1 is the intercept. At $d = 4$ the correct divisor is 95, not 96: $314/95 = 3.305$.

Solution — Extended Exercise 6.1 (3/3)

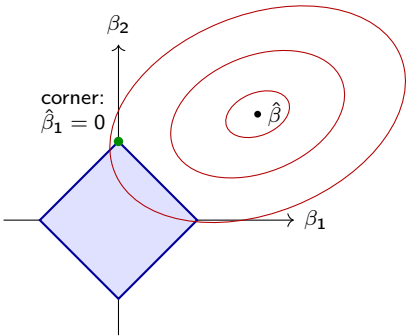
Solution

(2) Why they disagree. All three add a complexity penalty to the training fit, but with different strengths. BIC's penalty $\log(n) \approx 4.6$ per parameter is heaviest, so it stops earliest at the parsimonious $d = 2$. C_p/AIC use the lighter penalty 2, so they tolerate the small RSS drop $340 \to 322$ and pick $d = 3$. Adjusted R^2 penalises size only mildly and keeps improving to $d = 4$. Ordering by aggressiveness of shrinkage in model size, $\mathrm{BIC} \leq C_p \leq \mathrm{adj}\, R^2$ — exactly the pattern of ISLP Figure 6.2 on the Credit data.

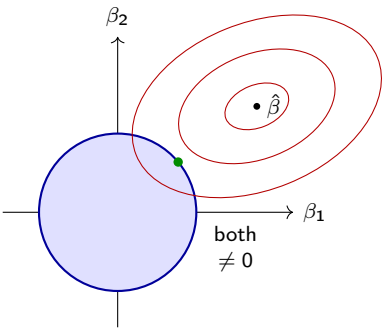
(3) Search cost. Best subset examines $2^p = 2^4 = 16$ models; forward stepwise fits $1 + \frac{p(p+1)}{2} = 1 + 10 = 11$. The gap is tiny here but explodes with p: at $p = 20$ it is $2^{20} \approx 10^6$ versus 211 — why best subset is infeasible past $p \approx 30$.

Why the lasso is sparse: the constraint geometry

Lasso (ℓ_1 diamond)



Ridge (ℓ_2 circle)



Takeaway

RSS contours first touch the ℓ_1 diamond at a *corner* ($\hat{\beta}_j = 0$); the smooth ℓ_2 circle almost never does.

ISLP Figure 6.7, redrawn

Exercise 6.1 — Counting Models in Subset Selection [Math]

Exercise (≈5 min) — try it before the solution

A regression problem has $p = 10$ candidate predictors.

1. How many models does *best subset* selection consider in total (including the null model)? Give the general formula and the number.
2. How many models does *forward stepwise* selection fit in total? Give the general formula and the number.
3. By roughly what factor does best subset exceed forward stepwise here, and why does the gap explode as p grows?

Live demo — §5 of the notebook

Switch to `chapter_06_lab.ipynb` §5 and run the counting cell: it prints $2^{19} = 524\,288$ best-subset fits against 191 for forward stepwise on Hitters, plus this slide's $p = 10$ case (1 024 vs. 56).

Solution — Exercise 6.1 (1/3)

Solution

(1) Best subset. For each size k there are $\binom{p}{k}$ models, and summing over all sizes gives

$$\sum_{k=0}^{p} \binom{p}{k} = 2^p.$$

For $p = 10$: $2^{10} = 1024$ models.

Solution — Exercise 6.1 (2/3)

Solution

(2) Forward stepwise. Start from the null model, then at step k (for $k = 0, \dots, p-1$) fit the $p - k$ models that add one predictor to the current model:

$$1 + \sum_{k=0}^{p-1} (p - k) = 1 + \frac{p(p+1)}{2}.$$

For $p = 10$: $1 + \frac{10 \cdot 11}{2} = 1 + 55 = 56$ models. (Backward stepwise fits the same count, $O(p^2)$.)

Solution — Exercise 6.1 (3/3)

Solution

(3) The gap. Here $1024/56 \approx 18\times$. Best subset grows *exponentially* (2^p) while forward stepwise grows only *quadratically* ($O(p^2)$). For $p = 20$, best subset already considers $2^{20} \approx 10^6$ models versus $1 + \frac{20 \cdot 21}{2} = 211$ for forward stepwise — which is why best subset is infeasible beyond $p \approx 30$ and stepwise heuristics are used instead.

Interpretation. The saving is bought with greed: forward stepwise commits to one predictor per step and walks a single path through the model lattice, so it can miss the best subset of a given size — its winner must still be validated by CV or a penalised criterion.

Common mistake

Dropping the null model from either count — both include it ($2^{10} = 1024$ subsets *including* $\varnothing$; $1 + 55 = 56$ stepwise fits, the leading 1 being M_0).

Extended Exercise 6.2 — Orthonormal Design: Ridge vs. Lasso [Math]

Extended exercise (≈15 min) — try it before the solution

Assume an *orthonormal* design, $\mathsf{X}^\top \mathsf{X} = I$, so the OLS estimates are $\hat{\beta}_j^{\mathrm{OLS}} = (\mathsf{X}^\top \mathsf{y})_j$ and the objective separates coordinate-by-coordinate.

1. **Ridge.** Minimise $\|\mathsf{y} - \mathsf{X}\beta\|^2 + \lambda\|\beta\|_2^2$ and show $\hat{\beta}_j^{\mathrm{R}} = \hat{\beta}_j^{\mathrm{OLS}}/(1+\lambda)$.
2. **Lasso.** Minimise $\|\mathsf{y} - \mathsf{X}\beta\|^2 + \lambda\|\beta\|_1$ and derive the soft-threshold solution $\hat{\beta}_j^{\mathrm{L}} = \mathrm{sign}(\hat{\beta}_j^{\mathrm{OLS}})\big(|\hat{\beta}_j^{\mathrm{OLS}}| - \lambda/2\big)_+$.
3. **Apply** both with $\lambda = 1$ to $\hat{\beta}^{\mathrm{OLS}} = (3.0, -1.5, 0.4, -0.2)$ and comment on shrinkage vs. selection.

Solution — Extended Exercise 6.2 (1/3)

Solution

Because $X^\top X = I$, $\|y - X\beta\|^2 = \text{const} + \sum_j (\beta_j - \hat\beta_j^{\text{OLS}})^2$, so every penalty separates across j.

(1) Ridge. Minimise $(\beta_j - \hat\beta_j^{\text{OLS}})^2 + \lambda\beta_j^2$. Setting the derivative to zero, $2(\beta_j - \hat\beta_j^{\text{OLS}}) + 2\lambda\beta_j = 0 \Rightarrow (1+\lambda)\beta_j = \hat\beta_j^{\text{OLS}}$, hence $\boxed{\hat\beta_j^{\text{R}} = \hat\beta_j^{\text{OLS}}/(1+\lambda)}$ — a constant *shrinkage factor*, never exactly zero.

(2) Lasso. Minimise $(\beta_j - \hat\beta_j^{\text{OLS}})^2 + \lambda|\beta_j|$. For $\beta_j > 0$, $2(\beta_j - \hat\beta_j^{\text{OLS}}) + \lambda = 0$ gives $\beta_j = \hat\beta_j^{\text{OLS}} - \lambda/2$ (valid while positive); symmetrically for $\beta_j < 0$. Combining and clamping at 0, $\boxed{\hat\beta_j^{\text{L}} = \text{sign}(\hat\beta_j^{\text{OLS}})(|\hat\beta_j^{\text{OLS}}| - \lambda/2)_+}$.

Solution — Extended Exercise 6.2 (2/3)

Solution

Why the clamp at zero? The penalty $\lambda|\beta_j|$ has a *kink* at $\beta_j = 0$. Away from zero calculus applies and gives $\beta_j = \hat{\beta}_j^{\mathrm{OLS}} \mp \lambda/2$; but when $|\hat{\beta}_j^{\mathrm{OLS}}| \leq \lambda/2$ each branch overshoots past zero and contradicts its own sign assumption, so the minimum sits *at the kink*: $\hat{\beta}_j^{\mathrm{L}} = 0$ exactly. A whole *interval* of OLS values, $[-\lambda/2, \lambda/2]$, maps to zero — that is why lasso solutions are sparse, while ridge's smooth penalty maps only the single point 0 to 0. (These are ISLP equations (6.14)–(6.15).)

(3) Apply with $\lambda = 1$ (ridge factor $1/(1+\lambda) = 0.5$, lasso threshold $\lambda/2 = 0.5$) — next slide.

Solution — Extended Exercise 6.2 (3/3)

Solution

$\hat{\beta}^{\mathrm{OLS}}$	3.0	-1.5	0.4	-0.2
$\hat{\beta}^{\mathrm{R}} = \hat{\beta}^{\mathrm{OLS}}/2$	1.5	-0.75	0.20	-0.10
$\hat{\beta}^{\mathrm{L}} = \mathrm{soft}(\cdot, 0.5)$	2.5	-1.0	0	0

Comment. Ridge multiplies *every* coefficient by the same factor 0.5: all four survive, merely shrunk. The lasso subtracts 0.5 from each magnitude and clamps at zero, so the two small coefficients ($0.4, -0.2$, below the threshold 0.5) are set *exactly to zero* while the large ones are shrunk by the same absolute amount. This is precisely why the ℓ_1 penalty performs *variable selection* and the ℓ_2 penalty only shrinks.

Common mistake

Thinking the lasso shrinks proportionally. It subtracts the same *absolute* amount from every coefficient; ridge is the proportional one — compare the 3.0 and 0.4 rows.